CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 38: Support Debugging and Operational Issue Resolution

Exam objective: Troubleshoot Claude Code configuration, MCP server failures, hooks, permissions, context issues, and tool execution failures, and monitor usage and operational behaviour to resolve developer-facing problems.

Q1 **A developer at a manufacturing company reports that Claude Code suddenly can no longer write to a file it could edit yesterday, with no error message explaining why — the edit silently appears to be skipped. What should the support engineer check first?**

A Whether the file has been renamed

B **Whether a permission rule (in managed, project, or local settings) now denies or asks for that tool/path, and whether a hook is silently blocking the write**

C Whether the developer's internet connection is stable

D Whether the file is a markdown file

CORRECT ANSWER **B**

WHY Silent blocking of an action that previously worked is a classic symptom of a permission rule change (deny/ask) or a hook intercepting the tool call, so checking the layered settings and any configured hooks is the correct first diagnostic step. File renames, connectivity, or file type are unrelated to this specific symptom.

Q2 **A team integrated a custom MCP server for internal ticketing, but Claude Code reports "MCP server unavailable" intermittently. What is the most systematic first troubleshooting step?**

A Immediately uninstall and reinstall Claude Code entirely

B **Check the MCP server's process/connection health, logs, and configuration (e.g., correct command/path, authentication, and whether it crashed or timed out) independent of Claude Code itself**

C Assume it is a permanent Claude Code bug and stop using MCP

D Ignore it since intermittent issues usually resolve themselves

CORRECT ANSWER B

WHY Since the MCP server is an external process, intermittent "unavailable" errors are most often caused by the server's own health, connectivity, auth, or configuration — isolating and checking those logs/config is the systematic approach before blaming the client. Reinstalling Claude Code or ignoring the issue skips proper root-cause diagnosis.

Q3 **A developer complains that Claude Code seems to "forget" earlier instructions partway through a long session and starts contradicting conventions established at the start. What is the most likely underlying operational issue?**

A. A corrupted installation requiring reinstall

B. Context window pressure — older context (including earlier instructions) may have been truncated or deprioritized as the session grew, requiring a context reset, summarization, or re-stating key instructions (e.g., via CLAUDE.md)

C. A permissions misconfiguration

D. An MCP server outage

CORRECT ANSWER B

WHY Long sessions can exhaust available context, causing earlier instructions to be dropped or deprioritized; the fix is managing context (resetting, summarizing, or reinforcing via persistent CLAUDE.md content) rather than reinstalling software. Permissions and MCP issues would produce access/tool errors, not a gradual drift in instruction-following.

Q4 A hook configured to run `npm test` after file edits appears to never execute, even though the configuration file looks correct on inspection. What should be checked next?

A **Whether the hook's trigger event name matches an actual supported lifecycle event, and whether the hook is defined in the settings file that is actually active for this session's precedence level**

B Whether the developer has a stable internet connection

C Whether `npm test` passes when run manually (this doesn't explain non-execution)

D Whether the project uses TypeScript or JavaScript

CORRECT ANSWER **A**

WHY A hook that never fires is typically due to a mismatched/incorrect trigger event name or being defined at a settings precedence level that isn't actually in effect for the current session (e.g., overridden or shadowed by another layer) — verifying both is the key diagnostic step. Internet connectivity and the project's language have no bearing on whether a local hook executes, and confirming the test command works manually doesn't address why the hook itself isn't firing.

Q5 **An enterprise's security team wants to retroactively investigate which tool calls and file accesses Claude Code performed during a specific incident window last week. What operational capability supports this kind of investigation?**

A There is no way to review historical tool usage

B **Reviewing available logs/audit trails of tool executions and permission decisions (where logging/monitoring has been enabled) to reconstruct the sequence of actions**

C Asking Claude Code to "remember" what it did, relying purely on model recall

D Reinstalling Claude Code to generate a fresh report

CORRECT ANSWER B

WHY Operational monitoring and audit logging of tool executions and permission decisions is the correct mechanism to reconstruct historical activity for an investigation, assuming logging was enabled. Relying on the model's own recall is unreliable for a security investigation, and reinstalling software doesn't produce historical records.

Q6 A developer reports that a Skill they expected to be used was never invoked during a relevant task, and Claude instead solved the problem in a generic way. What is a likely cause to investigate?

A **The Skill's trigger/description metadata doesn't clearly match the task context, so it wasn't selected/loaded, or the Skill isn't available in the current project/settings scope**

B Claude Code does not support Skills at all

C The developer's terminal font is incompatible

D The Skill ran successfully but logged nothing

CORRECT ANSWER **A**

WHY Skills are progressively loaded based on relevance signals (like their description/metadata) and availability in the current configuration scope; if those don't align with the task, the Skill won't be surfaced or used — this is the key thing to check. Claude Code does support Skills, and terminal font has no bearing on skill invocation.

Q7 A CI pipeline job running Claude Code non-interactively starts failing with a permission error it didn't have last week, and nothing in the pipeline's own YAML changed. What should the engineer investigate?

A **Whether managed or project settings were updated (e.g., by the platform team) to tighten permissions, affecting the service identity/credentials the CI job uses**

B Whether the YAML file has invalid syntax (ruled out since nothing changed)

C Whether the repository was renamed

D Whether the developer's local machine has a firewall issue (irrelevant to CI)

CORRECT ANSWER **A**

WHY Since the pipeline config itself didn't change, the most likely cause is an upstream change to managed or project-level permission settings that now restrict the CI service identity's actions — that's the first place to check. The other options are either explicitly ruled out by the scenario or irrelevant to a CI environment.

Q8 **A support engineer is debugging why a hook's shell script runs but seems to have no effect on blocking a risky tool call. What is a likely root cause to check?**

A **The hook script's exit code/output doesn't match what Claude Code interprets as a "block" signal (e.g., wrong exit code or missing required output format), so the block isn't being honored**

B The shell script file has a `.sh` extension

C The developer's shell prompt theme

D The repository has too many files

CORRECT ANSWER **A**

WHY Hooks communicate pass/block decisions through specific exit codes or output contracts; if the script doesn't conform to the expected signal, Claude Code won't treat the call as blocked even though the script executed. File extension, prompt theme, and repository size are unrelated to hook decision logic.

Q9 **A team notices unusually high token/API usage for a specific repository compared to others of similar size, driving up costs. What is the best operational approach to diagnose the cause?**

A Immediately disable Claude Code for all repositories

B **Review usage/monitoring data for that repository's sessions (e.g., context size per session, frequency of large file reads, repeated long-running sessions) to identify the specific driver of elevated usage**

C Assume it's expected and take no action

D Blame the developers without investigating data

CORRECT ANSWER **B**

WHY Usage monitoring data broken down by repository/session is the correct diagnostic tool to pinpoint specific drivers (e.g., oversized context, large file reads, long sessions) behind anomalous cost, enabling a targeted fix. Disabling the tool entirely, ignoring it, or blaming people without data are not evidence-based operational responses.

Q10 A developer's Claude Code session reports a tool execution failure for a bash command that works fine when run manually in their terminal. What is a key difference to investigate?

A **The environment/working directory, environment variables, or permission context under which Claude Code executes the command may differ from the developer's interactive shell session**

B The command must be fundamentally broken since it's the same text

C Claude Code cannot run bash commands under any circumstances

D The developer mistyped the command only in their manual test

CORRECT ANSWER **A**

WHY Tool execution happens in a context that may have different working directory, environment variables, or permission scoping than an interactive terminal, so comparing those execution contexts is the right diagnostic step when the same command behaves differently. Claude Code does support bash execution, so that premise is false, and assuming a typo only in the manual test isn't a reasoned diagnosis.

Q11 An organisation's managed settings were recently updated to add a new deny rule, and now multiple unrelated teams report broken workflows. What troubleshooting step should the platform team prioritize?

A **Roll back or carefully review the new managed settings deny rule for unintended overly-broad scope affecting more tools/paths than intended**

B Tell every team to individually work around the new restriction in their own project settings

C Assume the teams are misusing Claude Code

D Disable managed settings entirely going forward

CORRECT ANSWER **A**

WHY Since the symptom started right after a managed settings change and spans multiple unrelated teams, the most likely cause is an overly broad new rule — reviewing/rolling it back is the targeted fix. Telling teams to individually work around an org-level policy undermines the point of centralized managed settings, and abandoning managed settings entirely removes an essential governance control.

Q12 A developer reports that an MCP-provided tool used to return rich structured data but now returns truncated or malformed responses. Where should debugging start?

A **The MCP server's own logs and its handling of the specific request/response, including any recent server-side updates or rate limiting**

B The developer's CLAUDE.md file

C The Claude Code application's UI theme settings

D The developer's keyboard layout

CORRECT ANSWER **A**

WHY Since the MCP server is the external component producing the data, truncated/malformed responses point to investigating the server's own logs, recent updates, or rate-limiting/timeouts first. CLAUDE.md, UI themes, and keyboard layout have no relationship to MCP server response integrity.

Q13 After a recent Claude Code update, a team's previously working custom command now fails to parse correctly. What is the most structured approach to resolving this?

A Abandon custom commands permanently

B **Check the release notes/changelog for the update for breaking changes to command syntax or behavior, and compare the command's format against current documentation**

C Assume the file got corrupted by a virus

D Rewrite the entire project from scratch

CORRECT ANSWER **B**

WHY Software updates can introduce breaking changes to supported syntax/behavior, so checking release notes and comparing against current documentation is the structured way to identify and fix a regression tied to an update. Abandoning the feature or rewriting the project are disproportionate responses to what is likely a syntax/version mismatch.

Q14 **A finance team's Claude Code sessions in a shared CI environment occasionally hang indefinitely on a specific MCP tool call, consuming pipeline minutes. What operational safeguard should be added to prevent this from recurring?**

A Nothing — hangs are unavoidable and must be tolerated

B **Configure appropriate timeouts for MCP tool calls (or the pipeline step) combined with monitoring/alerting so hangs are caught and terminated automatically rather than consuming resources indefinitely**

C Remove the MCP server entirely with no replacement

D Increase the pipeline's resource quota so it can hang longer

CORRECT ANSWER **B**

WHY Adding timeouts and monitoring/alerting around MCP tool calls or pipeline steps is the correct operational safeguard to automatically catch and terminate hangs rather than letting them consume indefinite resources. Simply increasing quota masks the symptom rather than fixing it, and removing the MCP server entirely may eliminate needed functionality instead of addressing the reliability gap.

Q15 A developer insists Claude Code is "ignoring" their CLAUDE.md instructions, but investigation shows the file is correctly formatted and in the right location. What is another plausible explanation to investigate?

A. There may be a conflicting or more specific instruction elsewhere (e.g., a nested CLAUDE.md, a managed setting, or a hook) that overrides or contradicts the top-level instruction, or the instruction may have been pushed out of context in a long session

B. CLAUDE.md files are purely decorative and never affect behavior

C. The developer's operating system is incompatible with CLAUDE.md

D. CLAUDE.md only works for the first five minutes of a session

CORRECT ANSWER A

WHY When a correctly placed CLAUDE.md seems ignored, likely causes include a conflicting nested CLAUDE.md or managed-level override, a hook altering behavior, or context truncation in a long session pushing the instruction out — all are legitimate things to check. CLAUDE.md is a functional, not decorative, mechanism, and it has no OS dependency or arbitrary time limit.

Q16 A security team is reviewing the risk of running Claude Code non-interactively against an untrusted, externally-contributed repository (e.g., a third-party pull request checked out in CI). They are concerned that hooks, skills, custom commands, subagents, plugins, MCP servers, and memory files defined inside that untrusted repo could auto-load and execute with no prompt, creating a code-execution risk before anyone reviews the content. What operational mode mitigates this?

A Bare mode, which skips auto-discovery of hooks, skills, commands, subagents, plugins, MCP servers, and memory files for the run, so untrusted repository content can't silently execute

B Increasing the context window size for the session

C Adding another nested CLAUDE.md file inside the untrusted repo

D Switching the permission mode to "auto" so actions run with background checks

CORRECT ANSWER A

WHY Bare mode is designed exactly for this scenario — it disables automatic discovery and loading of repo-defined hooks, skills, commands, subagents, plugins, and MCP servers, which closes the gap where an untrusted repository's own configuration could otherwise execute without any prompt. Context window size is unrelated, adding another CLAUDE.md inside the untrusted repo only adds more untrusted content, and "auto" permission mode increases autonomous execution rather than restricting it.

Q17 A platform team traces a cost spike to engineers routinely running single Claude Code sessions for many hours without ever resetting context, causing each turn to resend a very large accumulated context. They want both a coaching fix for developers and a data-driven way to confirm which sessions/repositories are driving the spend. What combination addresses this specific pattern (distinct from simply "investigate usage data")?

A Coach developers to use `/clear` (reset context) and `/compact` (summarize/shrink context) during long sessions, combined with OpenTelemetry usage export or the Analytics API to quantify which sessions and repositories are accumulating the largest context/spend

B Permanently cap every session to five minutes regardless of task complexity

C Remove Claude Code access from the engineers with the highest usage, with no further diagnosis

D Tell engineers to open a brand-new session for literally every single message

CORRECT ANSWER A

WHY `/clear` and `/compact` are the specific, supported mechanisms for managing context bloat in long-running sessions, and pairing that coaching with OpenTelemetry/Analytics API usage data lets the platform team confirm the actual drivers of spend rather than guessing. An arbitrary hard time cap or a blanket new-session-per-message rule ignores legitimate task needs, and removing access without diagnosis doesn't identify or fix the root cause. --